

Problem 1: Self assessment.

In question 6 part (b), I believe my solution is an improvement over the provided one. Their function g is unnecessarily complicated by treating the case $k > |V|$ separately. Notice that if $k > |V|$, then (G, K_k) cannot be a subgraph isomorphism because K_k has more vertices than G . By simply defining $g(G, k) = (G, K_k)$, the logic is simplified while still capturing the necessary conditions. This streamlined approach demonstrates that my solution is both more concise and effective.

Problem 2: True, false, or unknown.**(a)**

If $P \neq NP$ and L is NP-complete, then $L \notin P$.

Proof.

Assume $P \neq NP$. Let L be NP-complete.

Assume for contradiction that $L \in P$.

Since L is NP-complete. This means that $L \in NP$ and every NP problem reduces to L .

Thus, let $A \in NP$. Thus, $A \leq_p L$. Since $L \in P$, by proven in class this implies that $A \in P$.

Therefore, $NP \subset P \implies P = NP$ which is a contradiction. Thus, $L \notin P$. ■

(b)

If $L \in NP$, then $L \in P$.

Proof.

This is unknown, since if it were true then this would imply $NP \subset P \implies NP = P$. If it were false then we know there is a $L \in NP$ s.t. $L \notin P \implies P \subsetneq NP \implies P \neq NP$. Both of these results are unknown. ■

(c)

Let V be an efficient verifier for some language $L \in NP$. Let f be the Cook-Levin mapping reduction from L to SAT. The function $f(x)$ selects a certificate c , runs $V(x, c)$, and construct a Boolean formula ϕ from its computational tableau.

Proof.

False.

The Cook-Levin reduction does not work by “selecting” a certificate c and running $V(x, c)$. Instead, it constructs a Boolean formula ϕ that is satisfiable if and only if there exists some certificate c that would make $V(x, c)$ accept. ■

Problem 3: Y's alternative reductions.

(a)

Require: An unweighted graph G

Ensure: A weighted graph G' and nonnegative integer k

```

1: function  $f(G = (V, E))$ 
2:    $V' \leftarrow V$ 
3:   Initialize  $G'$  to be a complete graph on  $V'$  and initialize all weights
   to  $\infty$  in  $E'$ 
4:   Initialize  $C$  to be an empty sequence of vertices
5:   for each permutation  $\Pi = (v_1, v_2, \dots, v_n)$  of  $V$  do
6:     if  $(v_i, v_{i+1}) \in E$  for all  $i = 1, \dots, n - 1$  then
7:        $C \leftarrow \Pi$ 
8:       break
9:   for each adjacent pair  $(v_i, v_{i+1})$  in  $C$  do
10:    Set the weight of  $(v_i, v_{i+1})$  to be 1 in  $E'$ 
11:  return  $(G' = (V', E'), k = |V|)$ 

```

Proof.

The reduction fails because it iterates over every permutation of the vertices, which takes factorial time. This factorial time complexity means the reduction does not run in polynomial time, so it does not serve as a valid polynomial-time mapping reduction. ■

(b)

- For each variable $x \in \phi$, create a variable gadget consisting of two vertices, respectively labeled by the literals x and $\bar{x}$, with *no edges between them*.
- For each clause $(\ell_i \vee \ell_j \vee \ell_k)$ in ϕ , create a clause gadget of three vertices, respectively labeled by these literals, with an edge between each pair of these vertices (i.e., a triangle).
- For each vertex of each clause gadget, create an edge between that vertex and the (unique) variable-gadget vertex having the same label.
- Finally, set $k = n + 2m$, where n, m are respectively the number of variables and clauses in ϕ .

Proof.

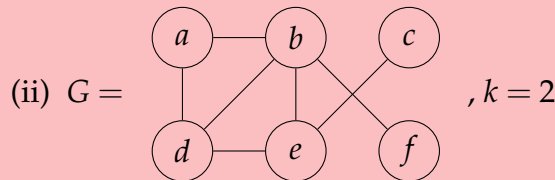
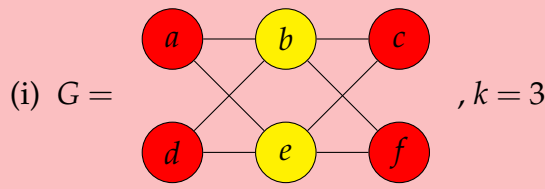
The mistake is that by removing the edge between the two literal vertices in each variable gadget, you lose the “forcing” mechanism that guarantees one of the two must be chosen in a minimum vertex cover. In the standard reduction, the edge between x and $\neg x$ forces the vertex cover to include at least one vertex from each variable gadget—ensuring that exactly one literal is “selected” per variable (since including both would exceed the budget). Without that edge, the reduction no longer forces a choice, and a vertex cover of size $n + 2m$ might be obtained by not “selecting” any literal for some variables. This breaks the correspondence between vertex covers and satisfying assignments for the 3SAT instance, rendering the reduction incorrect. ■

Problem 4: Scheduling exams with flying colors.

$$\text{SCHEDULING} = \{(S, C, k) : \exists f : C \rightarrow \{1 \dots k\} \text{ s.t. } f(C_i) = f(C_j) \implies C_i \cap C_j = \emptyset\}$$

(a)

Determine if each of the following is an instance of COLORING. If $(G, k) \in \text{COLORING}$, give a coloring of the graph. Otherwise, briefly explain why it's not possible.



Proof.

This configuration is not possible for case (ii) because if we assign one color to (a), then both (b) and (d) must use the only remaining color. However, since (b) and (d) are adjacent, they cannot share the same color, leaving us without a valid coloring option. ■

(b)

Show that $\text{COLORING} \leq_p \text{SCHEDULING}$. Since COLORING is NP-hard, SCHEDULING is NP-hard.

Proof.

Define: $g : \Sigma^* \rightarrow \Sigma^*$ s.t. $g(G = (V, E), k) = (S, C, k)$. Let $V = \{v_1, \dots, v_n\}$

We will define $S = E$, and $C = \{C_1, \dots, C_n\}$ where each $C_i = \{e \in E \mid e = (v_i, v) \text{ for some } v \in V\}$

Runtime:

This will take $O(|E|)$ to copy E into S . Then it will take $O(|V| + |E|)$ to construct C since we first make a size $|V|$ list then add $2|E|$ edges in total to the C_i 's. Thus, the whole function takes $O(|V| + |E|)$ which is polynomial with respect to $|G|$.

Correctness:

If G is k -colorable then suppose there is a valid coloring $c : V \rightarrow \{1, \dots, k\}$. This function just colors the vertices. Then consider $g(G, k) = (S, C, k)$.

Define: $f : C \rightarrow \{1, \dots, k\}$ s.t. $f(C_i) = c(v_i)$.

If $f(C_i) = f(C_j)$ for $i \neq j$ then this implies that $c(v_i) = c(v_j)$. This implies that v_i and v_j cannot be adjacent to each other thus $C_i \cap C_j = \emptyset$ since if it was not empty this would imply that either $(v_i, v_j) \in C_i \cap C_j$, but this is not possible since they are not adjacent.

If $g(G, k) = (S, C, k)$ is a valid scheduling then this implies there is a function

$f : C \rightarrow \{1, \dots, k\}$ s.t. $f(C_i) = f(C_j) \implies C_i \cap C_j = \emptyset$.

Define: $c : V \rightarrow \{1, \dots, k\}$ s.t. $c(v_i) = f(C_i)$.

Consider v_i, v_j s.t. $(v_i, v_j) \in E$, by definition of each C_i and C_j this would imply that $(v_i, v_j) \in C_i \cap C_j$. Thus, $f(C_i) \neq f(C_j) \implies c(v_i) \neq c(v_j)$. This implies that c is a valid coloring since for each adjacent vertices they are a different colors.

Thus, $\text{COLORING} \leq_p \text{SCHEDULING}$ ■

Problem 5: This recipe is bussin but NP-hard.

BUSSIN-RECIPE =

$$\{(D, m, p) : D \in \{0 \dots 10\}^{n \times n}, \exists S \subseteq \{1 \dots n\} \text{ s.t. } |S| \geq m \text{ and } \sum_{i,j \in S} D_{ij} \leq p\}$$

We assume for simplicity that $D_{ij} = D_{ji}$ and $D_{ii} = 0$ for all $i, j = 1, \dots, n$.

Show that BUSSIN-RECIPE is NP-hard.

Proof.

WTS: $\text{CLIQUE} \leq_p \text{BUSSIN-RECIPE}$

Define: $g : \Sigma^* \rightarrow \Sigma^*$ s.t. $g(G = (V, E), k) = (D, k, 0)$. Let $V = \{v_1, \dots, v_n\}$

We will let $D_{ii} = 0$, and $D_{ij} = D_{ji} = 0$ if $(v_i, v_j) \in E$ and $D_{ij} = D_{ji} = 10$ if $(v_i, v_j) \notin E$.

Runtime:

It will take $O(|V|^2)$ to make D since $|D| = |V|^2$, and the rest are $O(1)$ operations, assuming E is made with a set data structure. Thus, the runtime is $O(|V|^2)$ which is polynomial with respect to $|G|$.

Correctness:

If $(G, k) \in \text{CLIQUE}$ then we know we have a subset C of V s.t. for all $v, u \in C$ that $(v, u) \in E$, and $|C| \geq k$.

Therefore, we know that $g(G, k) = (D, k, 0)$. Since we know that for all pairs of vertices in C that they are connected. This implies that $D_{ij} = 0$ for all $v_i, v_j \in C$. Since C is at least size k we know we can just pick $S = \{i_1, \dots, i_k\}$ s.t. $v_{i_j} \in C$.

Therefore, since each $D_{ij} = 0$ we know that $\sum_{i,j \in S} D_{ij} = 0 = p$. Thus, $(D, k, 0) \in \text{BUSSIN-RECIPE}$

If $g(G, k) = (D, k, 0) \in \text{BUSSIN-RECIPE}$ this implies that there is a subset $S = \{i_1, \dots, i_r\}$ s.t. $|S| \geq k$. By definition of D since we know that each $D_{ij} = 0$ for $i, j \in S$. This implies that $(v_i, v_j) \in E$. Therefore, since we know that all pairs in S have $D_{ij} = 0$. This implies that all pairs of vertices v_i, v_j are connected in G . This is exactly the definition of a clique, and since we have at least size $|S| \geq k$. Thus, $(G, k) \in \text{CLIQUE}$.

Therefore, $\text{CLIQUE} \leq_p \text{BUSSIN-RECIPE}$ which implies BUSSIN-RECIPE is NP-Hard ■

Problem 6: Subset sum.

SUBSET-SUM = $\{(A, s) : A \text{ is a multiset of integers } \geq 0, \text{ and } \exists I \subseteq A \text{ s.t. } \sum_{a \in I} a = s\}$.

The reduction is as follows. As input it is given a 3CNF formula ϕ with n variables $x_1, \dots, x_n$ and m clauses $c_1, \dots, c_m$. (Recall that each clause is the OR of exactly three literals, where a literal is either some variable x_i or its negation $\bar{x}_i$.)

The reduction constructs a collection of numbers as follows:

1. For each clause c_j , it constructs integers $a_j = b_j$, which are $m + n$ decimal digits long (counting any leading zeros), whose j th digits are 1, and whose other digits are 0.
2. For each variable x_i , it constructs integers t_i and f_i , each $m + n$ decimal digits long, where:
 - (a) The $(m + i)$ th digits of both t_i and f_i are 1.
 - (b) The j th digit of t_i is 1 if literal x_i appears in clause c_j .
 - (c) The j th digit of f_i is 1 if literal $\bar{x}_i$ appears in clause c_j .
 - (d) All other digits of t_i and f_i are 0.
3. It constructs a target sum s that has $m + n$ decimal digits, where the first m digits are **[TODO: Part a]** and the last n digits are **[TODO: Part a]**.

The reduction outputs (A, s) , where A is the set of all the numbers a_j , b_j , t_i , and f_i .

(a)

Give the complete step 3 from the description above. You do not need to justify your answer.

The first m digits are 3, and the last n digits are 1.

(b)

The SUBSET-SUM instance for the given formula ϕ is therefore

$$A = \{10000, 10000, 1000, 1000, 10100, 1100, 1010, 10010, 10001, 1001\}, s = ?$$

Based on your answer in Part a, determine s in the example given above. Then, show that (A, s) is an instance of SUBSET-SUM by giving a subset of A that sums to s .

$$s = 33111$$

Consider $\{a_1, a_2, b_2, t_1, t_2, t_3\}$. $10000 + 01000 + 01000 + 10100 + 01010 + 10001 = 33111$. Thus, $(A, s) \in \text{SUBSET-SUM}$.

(c)

Using the reduction described above, construct the SUBSET-SUM instance corresponding to the 3SAT instance $\phi' = (x_1 \vee x_1 \vee x_1) \wedge (\bar{x}_1 \vee \bar{x}_1 \vee \bar{x}_1)$. (Note that ϕ' is not satisfiable.) Also state whether the output instance is in the language SUBSET-SUM.

Proof.

$$A = \{a_1, a_2, b_1, b_2, t_1, f_1\} = \{100, 10, 100, 10, 101, 11\}, s = 331.$$

$(A, s) \notin \text{SUBSET-SUM}$ since look at the last digit of s . It is one this implies that we must pick either t_1 or f_1 but not both.

If we pick t_1 then our current sum is 101. Thus, we need 230 more. However, this is not possible since the sum of $a_1 + a_2 + b_1 + b_2 = 220$, which is not enough to get to s .

(d)

Show that

$$\phi \in 3\text{SAT} \implies f(\phi) = (A, s) \in \text{SUBSET-SUM}.$$

Proof.

Assume $\phi \in 3\text{SAT}$. Let ϕ have n variables, and k clauses. This implies we have a function $\tau : \{x_1, \dots, x_n\} \rightarrow \{\text{True}, \text{False}\}$ that satisfies ϕ .

Consider $f(\phi) = (A, s)$.

$$A = \{a_1, a_2, \dots, a_k, b_1, \dots, b_k, t_1, f_1, \dots, t_n, f_n\}.$$

Consider the sum $S = \sum_{i=1}^n c_i$ s.t. $c_i = t_i$ if $\tau(x_i) = \text{True}$ and $c_i = f_i$ otherwise. Format it to

be a $k + n$ long digit with leading zeros if need be.

Since we only picked either t_i or f_i but not both this means that the $k + i$ spot is a 1, so for all $1 \leq i \leq n$ we have $k + i$ th spot is a one in S . Thus, we know that s and S are the same for this part.

Now consider the first k digits. Assume for contradiction that in the sum S spot j of the first k digits is zero. This would mean that clause for j , τ would not pick any $x_i, \bar{x}_i$ in the clause s.t. it was true. This is a contradiction of ϕ being satisfiable.

Therefore, we know that all of the first k digits are at least 1. The digits can also not be greater than 3. Since we know by definition that for the j th spot that there is only a max of three t_i 's f_i 's that have a one in that spot. Since if not this would imply there are more than three literals in a clause which is not possible.

Thus, we know that the digit is either 1,2,3. If it is one then add a_i, b_i to it. If it is two then add a_j to it.

Thus, we we add each a_i, b_i as neccessary we get that all the first k digits will now be 3 in our sum. Thus, we get the sum we want since the first k digits are 3, and the last n are 1.

Thus, $(A, s) \in \text{SUBSET-SUM}$ ■

(e)

Show that

$$\phi \in 3\text{SAT} \iff f(\phi) = (A, s) \in \text{SUBSET-SUM}.$$

Proof.

Assume $f(\phi) = (A, s) \in \text{SUBSET-SUM}$. Let τ have k clauses, and n variables.

Thus, we know there is a multiset $I \subset A$ s.t. $\sum_{a \in I} a = s$.

Consider the last n digits of s . These are all ones, so consider spot $k + i$. By definition of f , we know that only t_i, f_i have a one in the $k + i$ th place. This implies that either t_i , or $f_i \in I$ but not both for all i .

Therefore, define $\tau : \{x_1, \dots, x_n\} \rightarrow \{\text{True}, \text{False}\}$ s.t. $\tau(x_i) = \text{True}$ if $t_i \in I$ otherwise *False*.

Now consider the first k digits in s . Consider the j th one. The j th spot is 3. Assume for contradiction that for all f_a, t_b s.t. they have a one in the j th spot that they are not in I . This is not possible since the sum must be a 3 in that spot, but there are only two other numbers if we exclude the f_a, t_b . These are a_j, b_j . Thus, we know that at least one $f_a, t_b \in I$ s.t. they have a one in the j th spot.

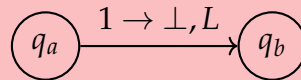
Now suppose for contradiction that τ does not satisfy ϕ . This means that for some clause c_j is not satisfied. Then by definition of τ this would imply that no t_a, f_b would

contribute to a positive value in the j th digit, but as shown above this is not possible. Therefore, τ does satisfy ϕ . Thus, $\phi \in 3\text{SAT}$ ■

Problem 7: Cook-Levin windows.

(a)

Specify *all* the valid 2×3 window contents that could appear in two adjacent rows of the tableau, if those rows represent the verifier TM taking the following transition:



s_1	s_2	s_3
s_1	s_2	s_3

q_a	1	s_3
s_1	$\perp$	s_3

s_1	q_a	1
q_b	s_1	$\perp$

s_1	s_2	q_a
s_1	q_b	s_2

s_1	s_2	s_3
s_1	s_2	q_b

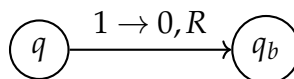
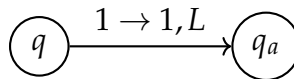
$s_1, s_2, s_3 \in \{0, 1, \perp\}$

(b)

Give an example by first defining one or more Turing machine transitions, then providing two adjacent rows where each 2×2 window is valid, but the transition is incorrect.

Proof.

Consider



Look at the tape's

Row i : ...1 q 11...

Row $i + 1$: ...1111...

All valid 2×2 Windows:

1	q
1	0

q	1
0	1

1	1
1	1

However, this is not a valid Turing machine since we get rid of the head. ■

(c)

Towards proving the Cook-Levin Theorem in lecture, we outlined a proof of the following statement:

Let language $L \in \text{NP}$ be arbitrary, and fix an efficient verifier VERIFYL for L . Given any instance x of L , in polynomial time we can construct an instance ϕ of SAT such that ϕ is satisfiable *if and only if* there exists a c such that the tableau of $\text{VERIFYL}(x, c)$ has q_{accept} in it.

Briefly explain why proving this statement proves that SAT is NP-hard.

Proof.

The statement shows that for any language L in NP and any instance x of L , we can transform x into a Boolean formula ϕ such that

$$x \in L \iff \exists c \text{ s.t. } \text{VERIFYL}(x, c) \text{ accepts.} \iff \phi \in \text{SAT}.$$

This transformation is computable in polynomial time. Since L was arbitrary, this means that every problem in NP can be reduced in polynomial time to SAT. By definition, this is exactly what it means for SAT to be NP-hard. ■

Problem 8: PSPACE and TQBF

(a)

Determine the base case when $t = 1$.

Proof.

When $t = 1$, the base case $\phi_{c_1, c_2, 1}$ states that machine M can go from configuration c_1 to configuration c_2 in at most one step. This can be captured by a Boolean condition that checks whether (1) c_1 is already an accepting configuration and thus equals c_2 , or (2) c_2 is exactly the successor configuration obtained by applying one step of M 's transition function on c_1 . In the second case, we verify that the tape contents, head position, and internal state of c_2 match the outcome of executing a single transition from c_1 . ■

(b)

Surprisingly, assuming the base case is correct, Y 's reduction is actually correct. However, it is not efficient. Briefly explain why.

Proof.

Because the naive recursive construction “unrolls” over all possible times up to T , and T can be as large as $2^{p(n)}$ for some polynomial p , the resulting formula can grow to exponential (or worse) size in n . In other words, writing down $\phi_{c_{\text{start}}, c_{\text{accept}}, T}$ explicitly requires $\Omega(T)$ steps just to construct all subformulas, which is not polynomial-time in the length of the input. ■

(c)

Explain how you would fix Y 's reduction so that it is efficient. Briefly justify its correctness and determine the size of $\phi_{c_{\text{start}}, c_{\text{accept}}, T}$.

Proof.

A more efficient approach modifies Y 's naive construction by introducing a *divide-and-conquer* style encoding of the “reachability within T steps” relation and using $\forall$ -quantified variables to avoid explicitly writing out subformulas for every time index. Specifically, instead of unrolling the statement “ M goes from c_1 to c_2 in $\leq t$ steps” for each time from 1 to T , we introduce a middle configuration c_m and say “there exists c_m such that M goes from c_1 to c_m in $\leq t/2$ steps and from c_m to c_2 in $\leq t/2$ steps.” Critically, rather than duplicating the subformulas for every (c_1, c_2, t) , we quantify over all

configurations and time values universally. In other words, we create a single “template” subformula $\phi_{c_1, c_2, t}$ and assert it holds *for all* valid triples (c_1, c_2, t) . This collapses the exponential blow-up that arises from naive recursion because we only write one copy of the subformula per “level” of the recursion, and we reuse it for all relevant triples by binding them with universal quantifiers.

This construction is both correct and efficient. It remains correct because halving the time repeatedly captures exactly all ways to reach c_2 from c_1 in up to T steps: a path of length T always decomposes into two paths of length $T/2$. Moreover, the universal quantification over (c_1, c_2, t) ensures that the subformula applies to *every* portion of the computation without needing to explicitly rewrite it each time. The size of the resulting QBF is polynomial in $|w|$ because: (1) each configuration’s description is polynomial in $|w|$; (2) the depth of the recursion is $O(\log T)$, and T itself is at most $2^{p(|w|)}$, so $\log T$ is polynomial in $|w|$; and (3) we do not replicate subformulas for every single time step but instead reuse them via universal quantifiers. As a result, the overall formula $\phi_{c_{\text{start}}, c_{\text{accept}}, T}$ can be constructed in time (and size) polynomial in $|w|$, making it a valid polynomial-time reduction. ■